美的ESCM钢材接口文档

一、环境及配置信息

测试环境地址：https://apiuat.midea.com/SDP/sdp-service/service/driver/data/in/
生产环境地址：https://api.midea.com/SDP/sdp-service/service/driver/data/in/

测试环境：

app_key：ESCM_A0015028
app_secret: fc2o2z4xy9dpmggovpxon6tma79niydb
供应商编码：A0015028
RSA公钥:

MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAgKQXm78nopJNxv0EaB43zQnVEzZoGbGB
ZeGSDA3/8wRWNZkV4Yshl6iDPkTOaqfrT1rTEN0EKNn233/Z7hBJuWYugFC6T3y0Z+ioJksYKoeBi84U
Mx5+63MYkZ7NMLaCw2aqJ2Ek/u0/C4QrTUQ02Vq7ieSez3LRfNeoOoU+0P9al0yV30qPnjtIyM72KB
wLkVEfMORA5rn9tB3NQZ+ykGD6lk1/KI58FI/uxff5pc0feMvFwkL9eIQg+KPUB8FqR1c2L7hhad/TS4a
YlJxZMj7mMYuYQQIoExkJQMRo0UxAEYrxoLOmC5e9mgB3Cd+TdckxEdCp3+cjqVM1Fkzf+wIDAQAB

RSA私钥:

MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQCApBebvyeikk3G/QRoHjfNCdUTN
mgZsYFl4ZIMDf/zBFY1mRXhiyGXqIM+RM5qp+tPWtMQ3QQo2fbff9nuEEm5Zi6AULpPfLRn6KgmSxg
qh4GLzhQzHn7rcxiRns0wtoLDZqonYST+7T8LhCtNRDTZWruJ5J7PctF816g6hT7Q/1qXTJXfSo+eO0jIz
vYoHAuRUR8w5EDmuf20Hc1Bn7KQYPqWTX8ojnwUj+7F9/mlzR94y8XCQv14hCD4o9QHwWpHVzY
vuGFp39NLhpiUnFkyPuYxi5hBAigTGQlAxGjRTEARivGgs6YLl72aAHcJ35N1yTER0Knf5yOpUzUWTN/
7AgMBAAECggEAWqJCN+tgI3xs7AwL9MeFjSKmSxkEE0Yq4vsgiPm8IR4nFzA6aGQ8ULvbbB3SSPbyJ
UDagIEXkyiudR2BD+fN6aACl0lJWwODn8od3oz+A8jhDv0s4i9UQNMgtkhqTS3px34n2ALrq+i/iH+ge
pnZmmIAy9CVy5cb3Zeb4U9LyLd/r8hjQBBNfUWHl3wyfJB0qEyen3JgNGunTLfI5vH0JrlEuRsFiKxUy7y
CNLLoDoSlKLcf3+2mEt89oh1lRkIeQxFGvI4EZjHEU+o41xr2AhQTlnLKCAORSMbvkGeEHWZYCdFOH
NFwL860moVVeS3gl0cSxv+CuRmpCtCZkSEbIQKBgQDJB3g5yHJU5LKw942fevq6JULN/SUFAXfRiLHd
mYgnslck7evYj8LM6kLL78KnjIaGwvmv4FCQQKoTPx7rhIwZXl3/yZs8ttM04Z6Jhfp+3phPbP7a4lASdF
pTqkdBhioOcaapEBxnXy4Anrr2CBrYlIJ/TDg3E1P4lXi6uVj1+QKBgQCj0UX9JaZ3ex80BsT7NtU42IcN/p
J4vqkFFKPNVWwp6roUAZnSTzO7AeM677pSrbPR6IErY3Dm5n/S4bK6smHPsHU/27msnveUvw1NBf
v8Kb8yNuRnFf2Z2TPHnDzhuB6ruusDug2b4F5nS0DNbQrJ0FRok4pbRBWiec2RoNUykwKBgQC/L5N
voOrbQgWPuF/AZL0/e2Bld3NVp2fEpzRBhFqxJjMqs/uxIkgpikGsooGFx2fJknTUh4kXJ+iYFVfatVxoZA8
RzTMOMvH7QPf94qWxMxG3t/lJQiLU6kNdfnv2HRqyUfQmpWPeC+88N7mnc13UzmH3Hw33kzIxID
aJnuy0QQKBgH5iGO35mxk42tTnmBb7sbp+iotIaLZQ2dM6njju+1jHZ7HBnmu7gPx4s1cn1gatTjrWd
rkDqfnpwJnwWFRjBMUXrQ2MNg4wQm/2iYtv330mtARIhyPkuld0U9de+TVnN/pjZablZlRo4EwuqZsS
0izq1gQsN6S7vTkxYWBFxdWTAoGADF8DnbEGo386/x4Ru957pm/qLorZbLyyAA+6pwfBuH6dw9r7d
OBN6Wfg/22c5RyZzAmYG+rJGpK9TlazxSd7XfM5zQum9QP/7Wddsxb0WZr4Osu/95b2Yi5Wfo22lYu
KxaMhukl9XBLR++WpmY09TvUrosNw3zvdjKUa0hJ2NAA=

二、接口清单

本系统	对方系统	方向	接口编号	功能接口名称	接口模式	调用方	接口提供方
ESCM	供方	ESCM->供方	SUPPLIER_A0015028_001	电子订单推送接口	推式	ESCM	供方
ESCM	供方	供方->ESCM	ESCM_SUPPLIER_002	电子订单接收接口	推式	供方	ESCM
ESCM	供方	ESCM->供方	SUPPLIER_A0015028_002	在产信息同步接口	拉式	ESCM	供方
ESCM	供方	ESCM->供方	SUPPLIER_A0015028_003	在运信息同步接口	拉式	ESCM	供方

三、供方接口

1、电子订单接收接口

推送环节：
1）提前批供应商确认中（63，供方显示提前批待确认）
2）供应商确认中（100，供方显示待确认）
3）供方最终确认中（130，供方显示待生效）

推送数据维度要求：

1）提前批供应商确认中、供应商确认中、供方最终确认中按分配单+供应商的维度推送（报文头汇总展示数据条数，供方接到所有数据再做展示）

（1）接口字段

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
1	batchNum	是	批次号	INT		参考以接口方为准
1	total	是	总数量	INT		
1	totalPage	是	总页数	INT		
1	pageSize	是	当前页数量	INT		
1	pageNum	是	当前页	INT		
1	datas	是	电子订单信息	数组对象		
2	uuid	否	宝钢拆分订单行唯一键	VARCHAR	32	
2	resultId	是	电子订单行ID	BIGINT	20	唯一匹配
2	distributeNo	是	资源分配单号	VARCHAR	30	
2	lineStatus	是	行状态(供方显示状态)	VARCHAR	10	
2	prebatch	是	提前批标识	VARCHAR	10	提前批标识(Y:提前批：N:正式批)
2	orderCode	是	电子订单行号	VARCHAR	30	
2	resultNo	是	分配结果行号	VARCHAR	30	
2	futuresMonth	是	期货月	VARCHAR	10	
2	categoryCode2	是	中类编码	VARCHAR	50	
2	categoryName2	是	中类	VARCHAR	100	
2	receiveAreaName	是	收货区域	VARCHAR	30	
2	categoryCode3	是	小类编码	VARCHAR	30	
2	categoryName3	是	小类	VARCHAR	100	
2	tradeMarkCode	是	牌号类型编码	VARCHAR	30	
2	tradeMarkName	是	牌号类型	VARCHAR	100	
2	itemCode	是	物料编码	VARCHAR	30	
2	itemName	是	物料描述	VARCHAR	300	

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
2	protocolNumber	是	技术协议号	VARCHAR	100	
2	grade	是	美的牌号	VARCHAR	30	
2	thickness	是	厚度	DOUBLE		
2	width	是	宽度	DOUBLE		
2	leng	否	长度	VARCHAR	255	
2	surfaceTreatment	否	表面处理	VARCHAR	30	热度锌才会有
2	galvanizedThickness	否	钢板镀层厚度	DOUBLE		热度锌才会有
2	surfaceGrade	是	表面等级	VARCHAR	30	
2	edgeProcessing	是	切边状态	VARCHAR	30	
2	annealingMode	是	退火方式	VARCHAR	30	
2	packingMode	是	包装方式	VARCHAR	30	
2	rollWeightPackingType	是	卷重/包装类型	VARCHAR	30	
2	rollOrPackageValue	是	卷重范围/包装重量	VARCHAR	100	
2	color	否	颜色	VARCHAR	30	
2	supplierMarkNo	是	供应商牌号	VARCHAR	30	
2	gbSurfaceTreatment	否	供方表面处理	VARCHAR	100	根据小类
2	gbSurfaceGrade	否	供方表面等级	VARCHAR	100	根据小类
2	gbCutType	是	供方切边状态	VARCHAR	100	
2	oilVolume	是	钢板单面涂油量	VARCHAR	100	
2	technology	是	工艺	VARCHAR	30	
2	demandTime	是	需求日期	DATETIME		
2	vendorCode	是	供应商编码	VARCHAR	30	
2	vendorName	是	供应商名称	VARCHAR	240	
2	brandPlaceName	是	品牌产地	VARCHAR	100	
2	portName	否	港口	VARCHAR	100	
2	paymentTypeName	否	*付款方式	VARCHAR	30	根据状态

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
2	arrivalArea	否	到货区域	VARCHAR	30	
2	quayShedName	是	码头仓	VARCHAR	100	
2	deliveryLocation	否	送货地点	VARCHAR	30	
2	shippingMethod	是	运输方式	VARCHAR	100	
2	shoppingMethod	是	提货方式	VARCHAR	100	
2	dispatchShipEstimatedDate	否	预计发船时间	DATETIME		
2	deliverPort	否	发货港	VARCHAR	240	
2	currencyCode	是	币种	VARCHAR	30	
2	taxRate	是	税率(%)	DECIMAL(10,8)		进口为0，国内一般13%
2	unit	是	单位	VARCHAR	5	
2	buyerAdjustQuantity	是	采购调整数量(吨)	DECIMAL(20,8)		
2	vendorDeadLineTime	是	供方确认截止时间	DATETIME		
2	vendorConfirmStatus	否	供方确认状态	VARCHAR	30	供方回传(30:接受;40:拒绝)
2	vendorRefuseReason	否	供方拒绝原因	VARCHAR	240	供方回传
2	vendorDesc	否	供方说明	VARCHAR	240	供方回传
2	vendorAdjustQuantity	否	供方调整数量(吨)	DECIMAL(20,8)		供方回传
2	vendorFeedbackPrice	否	供方反馈价格(含税)	DECIMAL(20,8)		供方回传
2	vendorAdjustVendorCode	否	供方调整供应商	VARCHAR	60	供方回传
2	vendorAdjustVendorName	否	供方调整供应商名称	VARCHAR	120	供方回传

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
2	vendorConfirmPeople	否	供方确认人员	VARCHAR	30	供方回传
2	vendorConfirmTime	否	供方确认时间	DATETIME		供方回传
2	gbCladdingWeight	否	供方钢板镀层重量（g/m2）	VARCHAR	50	
2	policyNo	否	追补政策号	VARCHAR	1000	追补后才有
2	reflectStand	否	追补体现标准	VARCHAR	60	追补后才有
2	vendorFinalConfirmStatus	否	供方最终确认状态	VARCHAR	30	供方确认后才有：30:接受
2	vendorFinalDesc	否	供方最终说明	VARCHAR	240	供方回传
2	puOrderNo	否	采购订单号	VARCHAR	50	订单生效后才会有
2	puOrderLineNo	否	采购订单行号	VARCHAR	30	订单生效后才会有
2	vendorOrderNo	否	供应商合同号	VARCHAR	240	供方回传
2	vendorOrderSubNo	否	供应商合同子项号	VARCHAR	240	供方回传
2	vendorFeedbackPriceSys	否	供方系统最终价格	DECIMAL(20,8)		供方回传
2	innerDiameterInfo	否	内径信息	VARCHAR	50	
2	innerDiameterValue	否	内径值	VARCHAR	50	
2	siliconSteelCoating	否	硅钢涂层	VARCHAR	50	
2	specialTechRequirement	否	特殊技术要求	VARCHAR	2000	

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
2	ifFeedbackPriceDiff	否	供方价格反馈-价格不一致标记	VARCHAR	10	N:不一致，Y:一致

（2）返回字段

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
1	status	是	返回状态：0成功，其他失败	INT		
1	msg	是		VARCHAR	500	

（3）发送参考

Y+OHlrUJq2LMUyoek1tVmDQmWcreFuvjDO3zf2/ojqHzzXoMpTdhluLxpslz8TGrOCRAWBq4yf9LCU8J
PAOcfFoZEiiTO+i/x6y8fvdYSVYoLGD9YONW4f4BQOePqp1nNZRXbkBBDOivatZWUYY3EYU5+xL/8kKd
JkSsCmHIaYigcr5stRMWgO48qYmxx94RxdDTRKBQVpQM7tw7BCDce+T9YBM3OtCa5BxG6SbxJOiA8qP9
2U8fMW1MGzNiiLTm5nB75pT476RVDWItBREfhSWUBi9up18TXOkufsPcGpo3X5L4DtRCiPiujZOGtOKC
zKITVyaYFLDp5XOnlOV4QUEEPQry8tQpiSyhcyalKuljDdO+OKO6T1urgLCOaRVOi2j3xnrO8vbbWzve
mBzskcsoPD1fiYRKJCGtBMxaIpr7utsDSxdy5bhP25DUus18xRAsj2Gfyc3vwGTTic/EIkIUzABOKXXZ
tAwflPg24kllGZOu7h64LO5+KAGsDwMpgkIJsBUkztTaglu2gOtIfmoLv+Tyw83FHu+Fq3lMtrhyga+r
4OOiTEvoiqeUzj/Psm7n9fq1sh/5qfg3BSJQWTf5/DO3fOyVk6VDSIiAPZXTOHKk44Yv25uE/ycoJ5Si
yQ1pKhZTWREN/rx7wY3RyFFY/VWGI56Q1rwzbggy2PcShZIQHsd5HhPPIbt9doDAAj1HFnbdKc6E/VgH
OevzQ8jj/G2Be8ladcg/s5PibLVLRYJl5eueegHpHeeDS3lat+eiTGRhla0zb74DDAWtxEnRRquJW528
uS4bJOAFTHSlM65w+OtopxtdIRqr2wb8bj/c9yx/brph3l8h4E4UX4BbucfE3lmHrujGUsa3b21iM2cg
Rz97aLhL/sKrg1KkPoLzLk3+kutP/NHk/W86/lGbzOmmODfhbORAyiNxOnDmq2xW32mJxAHhHLUCXMA3
BhQaT8mpOwAO4McQZpTCP13ZiyIh/kk2Csj5LXiqilR8G4+yhEbGsPTXZ1zsTWHqDqEgtz9rhqWS9Cwi
5n6MODj2wMl9MbTNAbwqoqbSg6oOLCQg+NsGXFc+cO25kCy54gBNm/luM2VqymnyTQWmGwWXZbixhfqZ
4mermd9z8npZj9Cs3Gq9KgTy2Y2hovitn2R8ctG9kpDPik07Hlz3leDzzglYfKObMopLDxPaSwqVZxXr
aNDE67P6zwK+XoaHMZr2D4jj4xDBQAPlj8jJAQMyrdMDlB2yLQkjDA2xK3acDvMsXAyCnO6wYeMoZ5xy
uDC2Ud5AK5NPglhFvy4XGMvPOoZqg4nfsDuGkqGISDQvYCVmYEvgNg5Qzh1KDexkT9MU13g7UomKvAH0
8z5vJADF/jugwJo3WnDQ+g5kNkmJnNIlDXPTNEnoxOSE1Yqy8Wg85CS2pAFQv1q0jLq7Tmwjvc34Dl7s
UIABIELoRjkjVrdk2hmHQuV/FdDQ1Op12pTEKOMxoG6WC04mADog+k8ZXXOKYegQqZ5u0eWrDHT2+1ci
7d8UmbNrSLI0+us6oakmqfxvA4PBSb7BcAhn5CxnJWfZU9mL4MeqnbQOCL/Y8n347A5GnlFqh3rpC89k
QxacNxJml8UqEpfi/LbK45TNZPfdvz5pQzITK2RC+apz6NhWQzu1n79MkYGWAJ/xmO8EPgXmjvjR2ZhJ
c8It/JoJ/LhRaF6Y2OVUM5k2dx4cEYmRXNdRifYkP6S1N7F+dkA6DaPbqtrkfalnmu58/l+ErglmBliO
MxZTmz3gVUy1OYKyzW2CJnYv1l/hDfQmiaNK4VO3z1ki936dTQpFqMppcm4gmEzXGoqSd287Y/8NPOqS
jh3TK7+g8GKumeJxOBhHZbOiVN23HvSry4FFqcR6cker2Qs6Fu1Ugbt1LCtEGkJby7Z3D5Zv9JTfhCJh
G8hqsiYD8JEXTNBGMeeHvquU5suja/WyFWng11e5k9FLBdcZelX4V9sznOpnTTYJ3VhkfNShfHwN3442
7R9vFtOqkpX+C1amS1sE5BXwYpS3Dsht+uYefINeyNZw4enWVcwpaxSE9EgZZymAJ2XviWUhrafa3Zng
QxASD9qHOOfLy7XT4C2VJZU83OdKkSlpnp6HvS/mPm3pAPAPMJfOXtRLeJkVvLKHLzOGNjJf7jwGdPxm
2dkXOPUUP8ZzQmMzn1wN/57VKsMz+DOaBG3fL9tpSdwrDIvxhSUvpeMBcMzZOeUZd18uoL+73WFKhHtD
DKsVfEa3UXvnIPTb9DGBJhl4GAMX6O5ED+Lvw5ZRYfRsXaUWQjmzQwCPbduv+wXeen/noPdwUUKT5xCi
cI+GPZvUAwhe9ZHHzYNT2qJFd27P5Bqbsl9Q2PShMkQ9i7wvx4f1Cv9q9HO7xgSCsgmdHQ/YLhBjXK71
UYMcBx61zYu+5rUQvKAdw+hoBquDiwjVZw/bYE8pZ5OvfBQvmzhZcnYjHX0K5gtKeYtaWqV4UY/j2+Uv
rNfqds+SGuXa4BkpWC6sOzwufKR5IJsNhrcZko4pbzq9IvKMXGKdi2/ZbbwOSh+KkAkIPfnRwoSy3/yi
qzIvmgTiSzeZTHZfWYQyD4hs6TJ/+U+ZrwFlIFShX6vcwLC1sM5yGcZdNT5Ou3+oH+y1eTyhNWqDP1vO
qZN1PBvdSkdXGpO38fOGgma3kHq55JOOqtcbq0z1QCFlkkohBt8vzAB+s+BS5VHaXwAvDnN8fFND9dT7
ale36PzzNx17rEVBdQRNCBOsfZ3aNBIOPRn9us9kvgJPyr0jKfOcaL4zX4at9ITBbYj0aiZacENmffdD
5U2KOlCP/FiWp5k8pCfpZsDHrsBwPnHmewdXYBDj/QiWqv5fFLIUsOIGxUxBrklzfQAYowBBjseYcVfD
98elHnY9dYQi5+QH5WZKQTTdlEpES6IREQ296/MCEuDNx5yTuvEz2MYs28Fi6XnptnVqUq5kQw25heSp
eAbXZQhsw9ncs3jCO1W1iAr+AwqOAHfHKDDMzlsDqAgTDrHRP4S4LQg3vJdmvy7WIJE22LANCL6mRRkc
5ykeOiqxdbgKPsAUNjG7dIO5hyJVcO2yJ3ULxGt/YGFTnxnyryRDOA3FeEdGg677fsdVoONOamrVHcEZ
eXJFVcdFs/CPg7f/UE+udFMTQPw4DWd+aBMzqywodOX1kIi/8a5Zctsxy4BAFXKUqIYmFOJgtYoyaH2O
D45uSGuUoztrDUx4FdYlaCsWX5MX8PoBRS5bn1Vb5BOOrjOMX3cjHXNFD6ZL94S3RiHW/alm/jyRstOB
QhOWymzlrLmGGFgmxf7CYMxeZppHLvztdmhmIE5LiDEP+GUwqG73CwJgggDcOKbytZ+WGg+rDA4RxZbg
8E4R4fYUZqtO8UvYCjCz1KVQB/GZkBILAqDGw7lRUbBmYwib3bTmXg==

(4) 返回参考

```
{
    "msg": "接收美的订单成功！",
    "status": 0
}
```

2、在产信息接口

同步频率：

每天同步一次

（1）接口字段

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
1	contractNum	是	合同号	VARCHAR	10	
1	factoryOrderNum	是	合同子项号	VARCHAR	20	

（2）返回字段

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
1	status	是	返回状态：0成功，其他失败	INT	10	
1	msg	是		VARCHAR	500	
1	result	是	返回对象集合			
2	contractNum	否	*合同号、钢厂资源号	VARCHAR(10)		
2	factoryOrderNum	否	*合同子项号	VARCHAR(20)		
1	orderStatus	否	合同状态	VARCHAR(20)		
1	prodName	否	*品类	VARCHAR(20)		
2	shopsign	否	*牌号	VARCHAR(50)		
2	specDesc	否	*规格	VARCHAR(50)		
2	znLayer	否	锌层	VARCHAR(100)		
2	surfaceGrade	否	表面处理	VARCHAR(100)		
2	applyStockWt	否	*材料申请在库量(吨)	DECIMAL(20,8)		
2	applyPassWt	否	*材料申请通过量(吨)	DECIMAL(20,8)		
2	steelmakingStockWt	否	*炼钢在库量(吨)	DECIMAL(20,8)		
2	steelmakingPassWt	否	*炼钢通过量(吨)	DECIMAL(20,8)		
2	hotRollStockWt	否	*热轧在库量(吨)	DECIMAL(20,8)		
2	hotRollPassWt	否	*热轧通过量(吨)	DECIMAL(20,8)		
2	siliconSteelStockWt	否	*硅钢常化在库量(吨)	DECIMAL(20,8)		
2	siliconSteelPassWt	否	*硅钢常化通过量(吨)	DECIMAL(20,8)		
2	picklingStockWt	否	*酸洗在库量(吨)	DECIMAL(20,8)		
2	picklingPassWt	否	*酸洗通过量(吨)	DECIMAL(20,8)		
2	coldRollingStockWt	否	*冷轧在库量(吨)	DECIMAL(20,8)		
2	coldRollingPassWt	否	*冷轧通过量(吨)	DECIMAL(20,8)		
2	annealingStockWt	否	*退火在库量(吨)	DECIMAL(20,8)		
2	annealingPassWt	否	*退火通过量(吨)	DECIMAL(20,8)		

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
2	hotDipStockWt	否	*热镀锌在库量(吨)	DECIMAL(20,8)		
2	hotDipPassWt	否	*热镀锌通过量(吨)	DECIMAL(20,8)		
2	readyStockWt	否	*准发确认在库量(吨)	DECIMAL(20,8)		
2	readyPassWt	否	*准发确认通过量(吨)	DECIMAL(20,8)		

（3）发送参考


```
Yus7OORjPPBVxp6R8OT52tY7o5I+A337lhnQyR9h6PZ11FOKyJfYzYKV893iD4nA958wqGjIC23BI2dv
IQkJ5nY1XL2eJKOSahBt9nSf3XxiNMiphUlmIEVBEFLVfpAPGMQbwBuXx1wK1IW17uJdLPEYNmz3tjCg
yyhkSaXz2HwuZXjTtJr9yRFr+m7OI2eHcga38tPyjugKPwiVdWp+htpfpg2zPfaH2uke9t4aOXhP9U5T
1ZuUH8lwGjyDXOeS/5WODeYMaa2POYpmsMvkpzoOP6mJeXOaOtaamyOv7tI3Av4MMRxPEnv1Pu/q1kFp
/b/fXr4qC8BeGeFxxBB2Jw==
```


（4）返回参考


```
{
    "msg": "查询美的生产跟踪成功！",
    "result":
"TY3WTxZHf4M++CcZXIOuJ2nXLzFP1FD5dXLJOIjFHX+jOQ3loSeQ2i+C/hJ4UMB7eu6pInFBInDz67L
kOI+eyA9MUoaYl2FIOiFI7yETG9+OOK/Djuyma7s7zHq41rCO5rwwaTIStMxhT3iTNFVuyd/yaf32ZmO
zHYMKi3TDPXn+NloT7qRvXWll7KMPrg16b5NdGVSQBl6EgizpGaMYmwl7jXm4G4isDWtuwDTFQTH4w+M
N9PIOBnfNMprsRxdWu8cF6VJclCZNsbq4D45Qf/+DDFxw3TUFKF4SOwFaP2eviOPEJec7OBaaOHYVS9A
cdOWzvVikgiwvQO1X69LcEA==",
    "status": 0
}
```


3、在运信息接口

同步频率：

每天同步一次

（1）接口字段

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
1	contractNum	是	合同号	VARCHAR	10	
1	factoryOrderNum	是	合同子项号	VARCHAR	20	

（2）返回字段

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
1	status	是	返回状态：0成功，其他失败	INT	10	
1	msg	是		VARCHAR	500	
1	result	是	返回对象集合	数据对象		
2	contractNum	否	*合同号/钢厂资源号（必填)	VARCHAR(10)		
2	factoryOrderNum	否	*合同子项号（必填)	VARCHAR(20)		
2	shopsign	否	*牌号（必填)	VARCHAR(50)		
2	specDesc	否	*规格（必填)	VARCHAR(50)		
2	znLayer	否	锌层	VARCHAR(100)		
2	surfaceGrade	否	表面处理	VARCHAR(100)		
2	packId	否	*母卷号（必填)	VARCHAR(50)		
2	netWeight	否	*材料净重（吨）（必填)	DECIMAL(20,8)		
2	grossWeight	否	*材料毛重（吨）（必填)	DECIMAL(20,8)		
2	shipName	否	船名/车号	VARCHAR(50)		
2	loadingPortName	否	启运港	VARCHAR(100)		
2	destPortName	否	目的港	VARCHAR(100)		
2	leaveDateCb	否	离港时间	VARCHAR(17)		

（3）发送参考


```
Yus7OORjPPBVxp6R8OT52tY7o5I+A337lhnQyR9h6PZ11FOKyJfYzYKV893iD4nA958wqGjIC23BI2dv
IQkJ5nY1XL2eJKOSahBt9nSf3XxiNMiphUlmIEVBEFLVfpAPGMQbwBuXx1wK1IW17uJdLPEYNmz3tjCg
yyhkSaXz2HwuZXjTtJr9yRFr+m7OI2eHcga38tPyjugKPwiVdWp+htpfpg2zPfaH2uke9t4aOXhP9U5T
1ZuUH8lwGjyDXOeS/5WODeYMaa2POYpmsMvkpzoOP6mJeXOaOtaamyOv7tI3Av4MMRxPEnv1Pu/q1kFp
/b/fXr4qC8BeGeFxxBB2Jw==
```

（4）返回参考


```
{
    "msg": "查询美的物流跟踪成功！",
    "result":
"dVNc7N5vwjb3ZvFUiuYg7/14a6meaTUXuxsBvpbyVboaJBqMPdcsXlIBrVVQGJEerEz76H3ux2lPK2q
NzPK58IbeOJ23S441XnTL3MjXnpFcj8pFu37X/8vx/Z7aRfGah4X2O53QzbJNqwRyI91xVm1zolsn4cO
nOAQNDQHzgXqiz9DUv6WTZ5HKfllD9bO8NFqusWAOF7szpGyRGrCq10fvLgOL9GWISn10djqyY+rCtYC
yAFsFNI6JjQuHIVQgnIO4l8kbdmKOHrSdfAga4JTD29Zc7VdAHphsPwaaZsK+Wp4iFrulAGqB2/jvO5H
SF5b0dOJ8xTyBGmVU6jmxTz6mZIjxkfiU3/b+dXSsCxg78TD13NlsojnE2noc3CX7u5XEXJcB8CrxIBl
xRHlCanh7SBSyx6n5ccVDe4G+YO4GbOIqChVXqf2bTtB/NONJdVgRE+Jegs6ymHZzUuQX81MdPBT4zGA
a9db0sBxunScmWwtzMphlre2tMOvQTM3CI7VeA6XL8KEGLlvm/Qz6kPbHjt6ZRVqacwSU3aD6nR/dWFL
DDwrzGggjO29B44pGalFDvTQJANp10Bv1o6S1TjPIlJcuuSgBB2WVt4MISSik9LDMyLP4YC7g/tQpm9+
eor7/1BwH6XmEh9tk5V7anJREFty+CFC4PfdBo4ELR5QaAqrZ71tcyrQrWeZHZv2khnoTO0NORvxvnzw
oEzqGNWhSSYsq+0p9LXGHAOXFhYBbvEWMv726wFCPucaMTvSCuigTHZWpL+1BAbJ1epDYGrmW/ebjbqk
aKiOl3DLxCDRLax8VXt4iCyHPdZxbo8DE55pynvusjL7MlOO/YRW4zm/UoomJPTL3LGR4RA35Xh3VmoG
2vTjkCA2YpW63dx4XtRSCr5KvXllhkXdhoIBIPojjzk2GoGUcBGItuCPxErzWiyWvGG3IMqTdo4/YzJT
Zsz2/VUF+GpFpO1CRCgdDJaA815qjgPZK6sxriz68V6xgidVWJTWpOQFxsKt9xvZg",
    "status": 0
}
```


四、ESCM接口

1、电子订单反馈接口

反馈环节：

反馈环节公共字段：
batchNum(必填)
total(必填)
totalPage(必填)
pageSize(必填)
pageNum(必填)
datas
resultId(必填)
resultNo(必填)

1）提前批供应商确认中（63，供方显示提前批待确认）

splitFlag（拆分需要）
lineNo（拆分需要）
uuid（拆分需要）
vendorConfirmStatus（必填）
vendorRefuseReason（拒绝40时必填）
vendorDesc（拒绝40时必填）
vendorAdjustQuantity（接受30时必填）
vendorAdjustVendorCode（选填）
vendorAdjustVendorName（选填）

vendorConfirmPeople（选填）
vendorOrderNo（接受30时必填）
vendorOrderSubNo（接受30时必填）
siliconSteelCoating（接受30时必填）
steelContractType（接受30时必填）

2）供应商确认中（100，供方显示待确认）

splitFlag（拆分需要）
lineNo（拆分需要）
uuid（拆分需要）
vendorConfirmStatus（必填）
vendorRefuseReason（拒绝40时必填）
vendorDesc（拒绝40时必填）
vendorAdjustQuantity（接受30时必填）
vendorAdjustVendorCode（选填）
vendorAdjustVendorName（选填）
vendorConfirmPeople（选填）
vendorOrderNo（接受30时必填）
vendorOrderSubNo（接受30时必填）
siliconSteelCoating（接受30时必填）
steelContractType（接受30时必填）
vendorFeedbackPrice（接受30时必填）

3）供方最终确认中（130，供方显示待生效）

vendorFinalConfirmStatus（必填）
vendorFinalDesc（选填）

4）最终价格回传（按需调用）
vendorFeedbackPriceSys（必填）

（1）SDP接口接入字段

层级	参数名称	参数	类型	长度	说明
1	businessId	SDP流水号	String	500	SDP-ESCM-供应商编码-001-时间戳+4位流水
1	requestObj	业务参数			
2	businessId	SDP流水号	String	500	SDP-ESCM-供应商编码-001-时间戳+4位流水
2	vendorCode	供应商编码	String	50	
2	data	业务数据	Object		Rsa加密整个data对象
1	systemCode	供方系统编码	String		根据文档提供
1	topicCode	主题编码	String		ESCM_SUPPLIER_001
1	targetSystemCode	系统编码			ESCM
1	timestamp	时间戳			yyyy-MM-dd HH:mm:ss
1	signMethod	SDP加密方式			Hmac
1	sign	SDP加密密钥			Hmac 加密后（app_key，app_secret，时间戳）参考第5点

（2）data加密字段

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
1	batchNum	是	批次号	INT		
1	total	是	总数量	INT		
1	totalPage	是	总页数	INT		
1	pageSize	是	当前页数量	INT		
1	pageNum	是	当前页	INT		
1	ifFeedbackPriceSys	否	是否最终价格标识	VARCHAR	2	Y:是,N:否
1	datas	是	电子订单信息	数组对象		
2	resultId	是	电子订单行ID	BIGINT		唯一匹配
2	resultNo	是	电子订单行号	VARCHAR	30	
2	splitFlag	是	是否拆分	VARCHAR	10	Y:是;N:否
2	lineNo	否	拆分行号	int	10	（1开始连续号）
2	uuid	否	宝钢拆分订单行唯一键	VARCHAR		
2	vendorConfirmStatus	否	供方确认状态	VARCHAR	30	供方回传(30:接受;40:拒绝)

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
2	vendorRefuseReason	否	供方拒绝原因	VARCHAR	240	供方回传(10：资源不足 20：价格差异 30：超出产线接单范围 40：其他) （拒绝时必填)
2	vendorDesc	否	供方说明	VARCHAR	240	供方回传(拒绝时必填)
2	vendorAdjustQuantity	否	供方调整数量(吨)	DECIMAL(20,8)		供方回传
2	vendorFeedbackPrice	否	供方反馈价格(含税)	DECIMAL(20,8)		供方回传
2	vendorAdjustVendorCode	否	供方调整供应商	VARCHAR	60	供方回传
2	vendorAdjustVendorName	否	供方调整供应商名称	VARCHAR	120	供方回传
2	vendorConfirmPeople	否	供方确认人员	VARCHAR	30	供方回传
2	vendorConfirmTime	否	供方确认时间	DATETIME		供方回传
2	vendorFinalDesc	否	供方最终说明	VARCHAR	240	供方回传
2	vendorOrderNo	否	供应商合同号	VARCHAR	240	供方回传

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
2	vendorOrderSubNo	否	供应商合同子项号	VARCHAR	240	供方回传
2	vendorFeedbackPriceSys	否	供方系统最终价格	DECIMAL(20,8)		供方回传
2	vendorFinalConfirmStatus	否	供方最终确认状态	VARCHAR	30	追补时回传(30:接受;40:拒绝)
2	siliconSteelCoating	否	供方硅钢涂层	VARCHAR	100	供方硅钢涂层
2	steelContractType	否	钢厂合同类型	VARCHAR	50	

(3) 返回字段

层级	字段名称	是否必填	字段描述	字段类型	字段长度	备注
1	status	是	返回状态：0成功，其他失败	INT		
1	msg	是		VARCHAR	500	

(4) 发送参考

```
{
  "businessId": "SDP-ESCM-A0015048-002-20260302-0005",
  "requestObj": {
```

```
    "data":
"CnqFeFzleu4wUes/N4uesqRFfGd5St7kZ/NT8ne/FXASyyRsEaJz801gqUWwgq44DWbSSlVNBt6zkgc
R17R+4m37i2qx1WJZ2h2asCpn3C77rX9BKdvdKCJO+45TIWd+nrIlRg+6UdcGXLv15kkeCGKz16iTjGA
EzWVcd6G4AbJrU/7CJEEzsGYaUcrwZk41OtpW2pfcbGkrAS7snvYpkQYG2rlXvadBGJ6uaDgxztV13pk
VNXDSI8tNYax9Xugyp4tBZsjsg93AtsKk79IxIIgukPr7JBvJAQtE4yRNRMTYrOSTunJOe+E6AoOrwQf
EHt/X2bWnUzNlHtevBGgOolEUiWh8KjAtxwxZ5HXEwy8VvnOBaHyE2dAR8KU6+4erlzoBnvSKhrn+Mu/
x7j72q5aYdSPO232Bu1g4uxUv8tpyNO9UX71M3rrPFp1My2ENwRjQjJT1fz6ocglorI8IdDtTTN/eOgD
sO1EtOoQspJa6gCvrxNCFvngiiwHWcn5w4kUG3HDWe9kTwx3BvIOlpvcYX8vt1zLQFk2beZEkmsMUwIv
XPpgoRRV64Boi2eDvkAzJRpvboRYFVfupG/GEw8CwyF7fjgS//igxNyORTtDl8SfzUeTIyOU6FDeJdhM
FltOmBwgTQaZ2pI1wWT6ktqVvRl3P3+5tVpuPltyEK2cXTeryYrnipydAquZfniTzdrph3PRCwBNRUxQ
uOq8bk6OyU4YU3/MVbpMGw5tp3Hnjlc+nUBE+B/2Hij1iv4LedLnoCYv5eRl9Z9VHNDtfotJVfIr6ykr
sTBjsmSjLBTzbW9myRXDs2v4BaSDZmiWeo1512VqqOckPTAZhun6yaRLbJ2t1PaEHWA9/TtDR6T976ba
8s4DxmO/A3DyjIEt+3s4rXDp5rlJk7OZYvPVGFwZEoQuVib6jGFfY/6Q4tQNM/AIMxKTemctCrDUFrNR
RyIclRAIXDmuU/kv7jIlCC6Vo5rgfABO6QNQfacVU/le7N/AprJUiqIWdSVQZwwPO",
    "businessId": "SDP-ESCM-A0015048-002-20260302-0005",
    "vendorCode": "A0015048"
  },
  "systemCode": "ESCM_A0015048",
  "topicCode": "ESCM_SUPPLIER_002",
  "targetSystemCode": "ESCM",
  "timestamp": "2026-03-02 14:44:55",
  "signMethod": "hmac",
  "sign": "46757FC2OD6404D74DC06590DD51C936"
}
```


（5）返回参考

```
{
    "status": 500,
    "msg": "[[供方确认状态为拒绝时供方拒绝原因/供方说明不能为空, 供方调整数量不能为空]]"
}
```

五、技术参考

1、Rsa加密类：

```
package com.midea.escm.common.utils;

import org.apache.commons.lang.ArrayUtils;

import javax.crypto.Cipher;
import java.io.ByteArrayOutputStream;
import java.nio.charset.StandardCharsets;
import java.security.*;
import java.security.interfaces.RSAPrivateKey;
```

```java
import java.security.interfaces.RSAPublicKey;
import java.security.spec.PKCS8EncodedKeySpec;
import java.security.spec.X509EncodedKeySpec;
import java.util.Base64;


public class RsaCryptoUtils {
    // 密钥大小
    private static final int KEY_SIZE = 2048;

    // 加密算法/模式/填充
    private static final String TRANSFORMATION = "RSA/ECB/PKCS1Padding";

    // 签名算法常量
    private static final String SIGN_ALGORITHM = "SHA256withRSA";

    // 密钥算法常量
    private static final String KEY_ALGORITHM = "RSA";

    //加密数据块Max(密钥模长2048)
    private static final int ENCRYPT_BLOCK_MAX_2048 = 245;

    //解密数据块Max(密钥模长2048)
    private static final int DECRYPT_BLOCK_MAX_2048 = 256;

    /**
     * 安全解密 - 必须先验证签名
     *
     * @param encryptedBase64  加密数据
     * @param signatureBase64  数据签名
     * @param publicKeyBase64  发送方公钥（用于验证签名）
     * @param privateKeyBase64 接收方私钥（用于解密）
     * @return 解密后的明文
     * @throws SecurityException 如果签名验证失败
     */
    public static String secureDecrypt(String encryptedBase64, String
signatureBase64, String publicKeyBase64, String privateKeyBase64) throws
Exception {

        // 1. 先验证签名
        if (!verifySignature(encryptedBase64, signatureBase64, publicKeyBase64))
{
            throw new SecurityException("数字签名验证失败，数据可能被篡改");
        }

        // 2. 签名验证通过后才进行解密
        return decrypt(encryptedBase64, privateKeyBase64);
    }

    /**
     * 安全加密 - 包含签名生成
     *
     * @param plainText               明文数据
     * @param senderPrivateKeyBase64  发送方私钥（用于签名）
     * @param receiverPublicKeyBase64 接收方公钥（用于加密）
     * @return String[] 包含加密数据和签名 [0]=加密数据, [1]=签名
     */
```

```java
    public static String[] secureEncrypt(String plainText, String
senderPrivateKeyBase64, String receiverPublicKeyBase64) throws Exception {
        // 1. 先加密数据
        String encryptedData = encrypt(plainText, receiverPublicKeyBase64);

        // 2. 对加密后的数据生成签名（保护加密数据的完整性）
        String signature = sign(encryptedData, senderPrivateKeyBase64);

        return new String[]{encryptedData, signature};
    }

    /**
     * 验证数据签名
     *
     * @param data            原始数据
     * @param signatureBase64 数据签名
     * @param publicKeyBase64 签名者公钥
     * @return 验证结果
     */
    public static boolean verifySignature(String data, String signatureBase64,
String publicKeyBase64) throws Exception {
        // 获取公钥
        RSAPublicKey publicKey = getPublicKey(publicKeyBase64);
        // 实例化签名对象
        Signature signature = Signature.getInstance(SIGN_ALGORITHM);
        // 初始化签名对象
        signature.initVerify(publicKey);
        // 更新签名数据
        signature.update(data.getBytes(StandardCharsets.UTF_8));
        // 验证签名
        return signature.verify(Base64.getDecoder().decode(signatureBase64));
    }

    /* 原有的基础方法保持不变 */

    /**
     * RSA 加密
     *
     * @param plainText       明文数据
     * @param publicKeyBase64 公钥
     * @return 加密后的数据
     */
    public static String encrypt(String plainText, String publicKeyBase64)
throws Exception {
        // 获取公钥
        RSAPublicKey publicKey = getPublicKey(publicKeyBase64);
        // 实例化加密对象
        Cipher cipher = Cipher.getInstance(TRANSFORMATION);
        // 初始化加密对象
        cipher.init(Cipher.ENCRYPT_MODE, publicKey);
        // 加密数据
        byte[] encryptedBytes =
cipher.doFinal(plainText.getBytes(StandardCharsets.UTF_8));
        // 返回加密后的数据
        return Base64.getEncoder().encodeToString(encryptedBytes);
    }

    /**
```

```java
     * 分段加密数据(密钥模长2048)
     *
     * @param data 明文数据
     * @param publicKeyBase64 公钥
     * @param isBase64Encoded 是否已Base64编码
     * @return
     * @throws Exception
     */
    public static String segmentEncrypt(String data, String publicKeyBase64,
boolean isBase64Encoded) throws Exception {
        // 实例化加密对象
        Cipher cipher = Cipher.getInstance(TRANSFORMATION);
        // 获取公钥
        RSAPublicKey publicKey = getPublicKey(publicKeyBase64);
        // 初始化加密对象
        cipher.init(Cipher.ENCRYPT_MODE,publicKey);
        // 分段加密
        byte[] enBytes = null;
        byte[] bytes = data.getBytes(StandardCharsets.UTF_8);
        byte[] b64 = isBase64Encoded ? Base64.getDecoder().decode(data) : bytes;
        for (int i = 0; i < b64.length; i += ENCRYPT_BLOCK_MAX_2048) {
            byte[] doFinal = cipher.doFinal(ArrayUtils.subarray(b64, i, i +
ENCRYPT_BLOCK_MAX_2048));
            enBytes = ArrayUtils.addAll(enBytes, doFinal);
        }
        return Base64.getEncoder().encodeToString(enBytes);
    }

    /**
     * RSA 解密
     *
     * @param encryptedBase64  加密数据
     * @param privateKeyBase64 私钥
     * @return 解密后的数据
     */
    public static String decrypt(String encryptedBase64, String
privateKeyBase64) throws Exception {
        // 获取私钥
        RSAPrivateKey privateKey = getPrivateKey(privateKeyBase64);
        // 解码加密数据
        byte[] encryptedBytes = Base64.getDecoder().decode(encryptedBase64);
        // 实例化解密对象
        Cipher cipher = Cipher.getInstance(TRANSFORMATION);
        // 初始化解密对象
        cipher.init(Cipher.DECRYPT_MODE, privateKey);
        // 解密数据
        byte[] decryptedBytes = cipher.doFinal(encryptedBytes);
        // 返回解密后的数据
        return new String(decryptedBytes, StandardCharsets.UTF_8);
    }

    /**
     * 分段解密
     *
     * @param data 密文数据
     * @param privateKeyBase64 私钥
     * @return
     * @throws Exception
```

```java
     */
    public static String segmentDecrypt(String data, String privateKeyBase64)
throws Exception {
        // 实例化解密对象
        Cipher cipher = Cipher.getInstance(TRANSFORMATION);
        // 获取私钥
        RSAPrivateKey privateKey = getPrivateKey(privateKeyBase64);
        // 初始化解密对象
        cipher.init(Cipher.DECRYPT_MODE, privateKey);
        //分块解密数据
        byte[] bytes = data.getBytes();
        byte[] decodeBytes = Base64.getDecoder().decode(bytes);
        int inputLen = decodeBytes.length;
        try (ByteArrayOutputStream out = new ByteArrayOutputStream()) {
            int i = 0;
            int offset = 0;
            byte[] buffer;
            while (inputLen - offset > 0) {
                if (inputLen - offset > DECRYPT_BLOCK_MAX_2048) {
                    buffer = cipher.doFinal(decodeBytes, offset,
DECRYPT_BLOCK_MAX_2048);
                } else {
                    buffer = cipher.doFinal(decodeBytes, offset, inputLen -
offset);
                }
                out.write(buffer, 0, buffer.length);
                i++;
                offset = i * DECRYPT_BLOCK_MAX_2048;
            }
            //返回解密后的数据
            return new String(out.toByteArray(), StandardCharsets.UTF_8);
        }
    }

    /**
     * RSA 签名
     *
     * @param data             数据
     * @param privateKeyBase64 私钥
     * @return 签名
     */
    public static String sign(String data, String privateKeyBase64) throws
Exception {
        // 获取私钥
        RSAPrivateKey privateKey = getPrivateKey(privateKeyBase64);
        // 实例化签名对象
        Signature signature = Signature.getInstance(SIGN_ALGORITHM);
        // 初始化签名对象
        signature.initSign(privateKey);
        // 更新签名数据
        signature.update(data.getBytes(StandardCharsets.UTF_8));
        // 返回签名
        return Base64.getEncoder().encodeToString(signature.sign());
    }

    /**
     * 获取公钥
     *
```

```java
     * @param publicKeyBase64 公钥
     * @return 公钥对象
     */
    public static RSAPublicKey getPublicKey(String publicKeyBase64) throws
Exception {
        // 清理公钥
        String cleanKey = publicKeyBase64
                .replace("-----BEGIN PUBLIC KEY-----", "")
                .replace("-----END PUBLIC KEY-----", "")
                .replaceAll("\\s+", "");

        // 解码公钥
        byte[] keyBytes = Base64.getDecoder().decode(cleanKey);
        // 实例化公钥规范
        X509EncodedKeySpec spec = new X509EncodedKeySpec(keyBytes);
        // 实例化密钥工厂
        KeyFactory keyFactory = KeyFactory.getInstance("RSA");
        // 生成公钥
        return (RSAPublicKey) keyFactory.generatePublic(spec);
    }

    /**
     * 获取私钥
     *
     * @param privateKeyBase64 私钥
     * @return 私钥对象
     */
    public static RSAPrivateKey getPrivateKey(String privateKeyBase64) throws
Exception {
        // 清理私钥
        String cleanKey = privateKeyBase64
                .replace("-----BEGIN PRIVATE KEY-----", "")
                .replace("-----END PRIVATE KEY-----", "")
                .replaceAll("\\s+", "");

        // 解码私钥
        byte[] keyBytes = Base64.getDecoder().decode(cleanKey);
        // 实例化私钥规范
        PKCS8EncodedKeySpec spec = new PKCS8EncodedKeySpec(keyBytes);
        // 实例化密钥工厂
        KeyFactory keyFactory = KeyFactory.getInstance("RSA");
        // 生成私钥
        return (RSAPrivateKey) keyFactory.generatePrivate(spec);
    }

    /**
     * 生成密钥对
     *
     * @return 密钥对
     */
    public static String[] generateKeyPairBase64() throws Exception {
        // 实例化密钥对生成器
        KeyPairGenerator keyPairGenerator = KeyPairGenerator.getInstance("RSA");
        // 初始化密钥对生成器
        keyPairGenerator.initialize(KEY_SIZE);
        // 生成密钥对
        KeyPair keyPair = keyPairGenerator.generateKeyPair();
```

```java
        // 获取公钥和私钥
        RSAPublicKey publicKey = (RSAPublicKey) keyPair.getPublic();
        RSAPrivateKey privateKey = (RSAPrivateKey) keyPair.getPrivate();

        // 返回密钥对
        return new String[]{publicKeyToBase64(publicKey),
privateKeyToBase64(privateKey)};
    }

    /**
     * 公钥转换为 Base64
     *
     * @param publicKey 公钥
     * @return 公钥 Base64
     */
    public static String publicKeyToBase64(RSAPublicKey publicKey) {
        // 获取公钥编码
        byte[] encoded = publicKey.getEncoded();
        // 返回公钥 Base64
        return "-----BEGIN PUBLIC KEY-----\n" +
formatBase64(Base64.getEncoder().encodeToString(encoded)) + "\n-----END PUBLIC
KEY-----";
    }

    /**
     * 私钥转换为 Base64
     *
     * @param privateKey 私钥
     * @return 私钥 Base64
     */
    public static String privateKeyToBase64(RSAPrivateKey privateKey) {
        // 获取私钥编码
        byte[] encoded = privateKey.getEncoded();
        // 返回私钥 Base64
        return "-----BEGIN PRIVATE KEY-----\n" +
formatBase64(Base64.getEncoder().encodeToString(encoded)) + "\n-----END PRIVATE
KEY-----";
    }

    /**
     * 格式化 Base64
     *
     * @param base64 Base64
     * @return 格式化后的 Base64
     */
    private static String formatBase64(String base64) {
        // 格式化 Base64
        return base64.replaceAll("(.{64})", "$1\n").trim();
    }

    // 测试安全流程
    public static void main(String[] args) throws Exception {
        // 生成两对密钥（发送方和接收方）
        String[] senderKeys = generateKeyPairBase64();
        String[] receiverKeys = generateKeyPairBase64();
        String senderPrivateKey = senderKeys[1];   // 发送方私钥
        String senderPublicKey = senderKeys[0];   // 发送方公钥
        String receiverPrivateKey = receiverKeys[1]; // 接收方私钥
```

```
        String receiverPublicKey = receiverKeys[0]; // 接收方公钥
        System.out.println("发送方私钥: " + senderPrivateKey);
        System.out.println("发送方公钥: " + senderPublicKey);
        System.out.println("接收方私钥: " + receiverPrivateKey);
        System.out.println("接收方公钥: " + receiverPublicKey);
    }
}
```

2、SDP接入Hmac加密参考

```
public class TMSSignUtil {

    /**
     * 对参数进行加密，获取API签名结果：sign ;
     * 推荐使用sha256算法，不推荐使用md5算法
     * @param params appKey timestamp signMethod
     * @param secret appSecret
     * @param signMethod hmac md5  hmac-sha256
     * @return
     * @throws IOException
     */
    public static String signTopRequest(Map<String, String> params, String
secret, String signMethod) throws IOException {
        // 第一步：检查参数是否已经排序
        String[] keys = params.keySet().toArray(new String[0]);
        Arrays.sort(keys);

        // 第二步：把所有参数名和参数值串在一起
        StringBuilder query = new StringBuilder();
        // 第二步：把所有参数名和参数值串在一起
        if (Constants.SIGN_METHOD_HMAC_SHA256.equals(signMethod)) { //签名的摘要算
法，可选值为：hmac，sha256
            query.append(secret);
        }
        for (String key : keys) {
            String value = params.get(key);
            if (StringUtils.isNotBlank(key) && StringUtils.isNotBlank(value)) {
                query.append(key).append(value);
            }
        }

        // 第三步：使用MD5/HMAC/SHA256加密
        byte[] bytes;
        if (Constants.SIGN_METHOD_HMAC_SHA256.equals(signMethod)){
            query.append(secret);
            bytes = encryptSHA256(query.toString());
        } else {
            bytes = encryptHMAC(query.toString(), secret);
        }

        // 第四步：把二进制转化为大写的十六进制(正确签名应该为32大写字符串，此方法需要时使用)
        return byte2hex(bytes);
    }

    /**
```

```java
     * SHA256算法加密
     * @param data
     * @return
     * @throws IOException
     */
    public static byte[] encryptSHA256(String data) throws IOException {
        byte[] bytes = null;
        try {
            MessageDigest messageDigest = MessageDigest.getInstance("SHA-256");
            messageDigest.update(data.getBytes(Constants.CHARSET_UTF8));
            bytes = messageDigest.digest();
        } catch (GeneralSecurityException gse) {
            throw new IOException(gse.toString());
        }
        return bytes;
    }




    public static byte[] encryptHMAC(String data, String secret) throws
IOException {
        byte[] bytes = null;
        try {
            //          CHARSET_UTF8  "utf-8"
            SecretKey secretKey = new
SecretKeySpec(secret.getBytes(Constants.CHARSET_UTF8), "HmacMD5");
            Mac mac = Mac.getInstance(secretKey.getAlgorithm());
            mac.init(secretKey);
            bytes = mac.doFinal(data.getBytes(Constants.CHARSET_UTF8));
        } catch (GeneralSecurityException gse) {
            throw new IOException(gse.toString());
        }
        return bytes;
    }

    public static String byte2hex(byte[] bytes) {
        StringBuilder sign = new StringBuilder();
        for (int i = 0; i < bytes.length; i++) {
            String hex = Integer.toHexString(bytes[i] & 0xFF);
            if (hex.length() == 1) {
                sign.append("0");
            }
            sign.append(hex.toUpperCase());
        }
        return sign.toString();
    }


    public static void main(String[] args) {
        String appKey = PropertiesUtil.getTMSPropertiesMdpBy("tms_appKey");
        String appSecret = PropertiesUtil.getTMSPropertiesMdpBy("tms_secret");
        String signMethod = "hmac-sha256";
        SdpSignUtils sdpSignUtils = new SdpSignUtils();

        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
        Date date = new Date();
        String timestamp = sdf.format(date);
```

```
        Map<String, String> params = new HashMap<>();
        params.put("timestamp", timestamp);
        params.put("signMethod", signMethod);
        params.put("appKey", appKey);
        String sign = null;
        try {
            sign = sdpSignUtils.signTopRequest(params, appSecret, signMethod);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }

        System.out.println(sign);
    }
}

/**
 * 加密参考
 */
public class TMSRequestService {

    private static String appKey =
PropertiesUtil.getTMSPropertiesMdpBy("app_key");
    private static String appSecret =
PropertiesUtil.getTMSPropertiesMdpBy("app_secret");
      public static void main(String[] args) {
        OkHttpClient client = new OkHttpClient();

        String timestamp = "2025-08-12 13:36:35";
        String signMethod = "hmac-sha256";
        String sign = null;//签名
        Map<String, String> SignParams = new HashMap<>();
        SignParams.put("timestamp", timestamp);
        SignParams.put("signMethod", signMethod);
        SignParams.put("appKey", appKey);
        try {
            sign = TMSSignUtil.signTopRequest(SignParams, appSecret,
signMethod);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
}
```